

BiolInsight: A 14-Day Learning Plan for a Multi-Agent Biological Data Platform

The Project

BiolInsight is a platform that analyzes biological data of multiple kinds — DNA/protein sequences, tabular lab/clinical data, and gene-expression/omics datasets — using 10 AI agents working in 5 sections. Each section has 2 agents with genuinely different jobs that hand off work to each other.


BiolInsight Platform
└─ Section 1: Data Intake
Agent 1 — File Detective: figures out what kind of file was uploaded
Agent 2 — Data Cleaner: fixes/flags missing or malformed data
└─ Section 2: Sequence Analysis (genomics)
Agent 3 — Sequence Reader: parses DNA/protein sequences, computes basic stats
Agent 4 — Pattern Finder: finds motifs/patterns, compares sequences
└─ Section 3: Statistical & Omics Analysis
Agent 5 — Stats Analyst: descriptive statistics, correlations on tabular data
Agent 6 — Anomaly Spotter: flags outliers / unusual values
└─ Section 4: Research Assistant
Agent 7 — Literature Scout: searches for relevant scientific info
Agent 8 — Summarizer: turns findings into plain-language notes
└─ Section 5: Reporting Dashboard
Agent 9 — Chart Maker: turns analysis results into visual charts
Agent 10 — Report Writer: combines everything into one final report, answers questions

Every section is a **self-contained mini-agent-system** you build and test on its own. Only in the final days do you wire them together into one platform with a single dashboard.

Why 14 days, not 7: 7 days is enough to understand *one* agent calling *one* tool. This project needs you to understand 10 agents, how they pass data between each other, real data-science libraries (pandas, biopython), a real backend, and a real dashboard — with genuine understanding, not copy-pasting. 14 days at 2–4 hrs/day is a realistic, honest estimate for a total beginner to get there solidly. If you want, we can also stretch this to 21 days later with zero pressure — better to actually understand it than rush it.

The Roadmap

Days	Phase	What you build
1–2	Foundations (done )	VS Code, Python basics — you've already completed this
3–4	LLM API + your first real agent	One working agent, deeply understood
5–6	Section 1: Data Intake	2 agents that detect and clean uploaded files
7	Section 2: Sequence Analysis	2 agents working with DNA/protein data
8	Section 3: Stats & Omics Analysis	2 agents working with tabular data
9	Section 4: Research Assistant	2 agents that search and summarize
10	Section 5: Reporting	2 agents that chart and write reports

Days	Phase	What you build
11	Agent-to-agent communication pattern	How sections hand off data to each other
12–13	Full backend + dashboard	FastAPI wiring all 10 agents + Streamlit UI
14	Integration, testing, and what's next	Full platform running end-to-end

Day 3 — Calling an LLM API (Deeply Explained)

Concept: what is an API, really?

An API (Application Programming Interface) is just a way for your code to "talk" to a service running somewhere else — in this case, Anthropic's servers running Claude. You send a request (a message), and you get a response back. It's the same idea as a website form: you fill it in, submit it, and get a result — except here it's code talking to code instead of you clicking buttons.

Commands, explained line by line

```
❏  
powershell  
pip install anthropic python-dotenv
```

`pip` = Python's package installer. It downloads code other people wrote so you don't have to write it yourself.

`install` = the action you want pip to take.

`anthropic` = the official Python library for talking to Claude's API.

`python-dotenv` = a small library for reading secret values (like API keys) from a `.env` file instead of hardcoding them in your script.

Setting up your `.env` file

Create a file literally named `.env` in your project root:

```
❏  
ANTHROPIC_API_KEY=your-key-here
```

Why not just paste the key directly into your Python file? Because if you ever upload your code to GitHub, anyone could see your key and use your account. Keeping it in `.env` (and telling git to ignore that file) keeps it private.

The code, explained piece by piece

```
❏  
python  
import os  
from dotenv import load_dotenv  
import anthropic  
  
load_dotenv() # reads .env and loads ANTHROPIC_API_KEY into environment  
  
client = anthropic.Anthropic() # automatically picks up ANTHROPIC_API_KEY  
  
response = client.messages.create(  
    model="claude-sonnet-4-6",  
    max_tokens=500,  
    system="You are a concise, helpful assistant.",  
    messages=[  
        {"role": "user", "content": "Explain what an API is in 2 sentences."}  
    ]  
)  
  
print(response.content[0].text)
```

`import os` — brings in Python's built-in tools for interacting with your operating system (files, environment variables). We don't use `os` directly much here, but `dotenv` relies on it internally.

`from dotenv import load_dotenv` — imports just the one function we need from the `dotenv` library, instead of the whole library.

`import anthropic` — brings in the whole library for talking to Claude.

`load_dotenv()` — this single line reads your `.env` file and makes `ANTHROPIC_API_KEY` available as an environment variable, invisibly, for the rest of the script.

`client = anthropic.Anthropic()` — creates a "client" object: think of it as opening a phone line to Anthropic's servers. It automatically looks for `ANTHROPIC_API_KEY` in your environment — that's why we loaded the `.env` first.

`client.messages.create(...)` — this is the actual API call. Every argument inside matters:

`model="claude-sonnet-4-6"` — tells Anthropic which version of Claude to use.

`max_tokens=500` — a *token* is roughly a chunk of a word. This caps how long the reply can be (about 375 words max here), which controls cost and response length.

`system="..."` — instructions that shape *how* the model behaves for the whole conversation (tone, role, constraints) — the model treats this differently from a normal message.

`messages=[...]` — a list of the conversation so far. Each entry is a dictionary with `"role"` (who's speaking — `"user"` or `"assistant"`) and `"content"` (what they said).

`response.content[0].text` — the API always returns a **list** of content blocks (because a reply could contain text *and* other things, like tool calls). `[0]` grabs the first block, and `.text` grabs the actual text out of it.

Exercise

Change the `system` prompt to: `"You are BioInsight, an assistant specialized in biology and lab data."` Ask it a biology question and see how the framing shifts its answer style. This is the exact same pattern your 10 agents will each use — only the `system` prompt and the tools available will differ per agent.

Day 4 — Your First Real Agent: Tool Use, Explained

Concept: what makes something an "agent" and not just a chatbot?

A chatbot answers with words. An **agent** can *take actions* — call real functions in your code (like reading a file, doing math, searching the web) — and then use the *result* of that action to form its final answer. The model doesn't run your code itself; it tells you "I want to call this function with these arguments," and your Python code is the one that actually runs it.

The code, explained

```
python
def get_weather(city):
    return f"The weather in {city} is sunny and 30°C."

tools = [
    {
        "name": "get_weather",
        "description": "Get the current weather for a city",
        "input_schema": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "City name"}
            },
            "required": ["city"]
        }
    }
]
```

`def get_weather(city):` — this is a normal Python function. The model will never run this itself — your code will, when told to.

`tools = [...]` — this is a description, written in a very specific format Claude understands, that tells the model "here's a tool you're allowed to ask for." It does **not** contain the actual function — just a description of its name, purpose, and what inputs it needs.

`"input_schema"` — this is JSON Schema, a standard way of describing the *shape* of data. `"type": "object"` means the input is a dictionary; `"properties"` lists what keys it should have; `"required"` lists which keys are mandatory.

```

python
messages = [{"role": "user", "content": "What's the weather like in Jeddah?"}]

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=500,
    tools=tools,
    messages=messages
)

for block in response.content:
    if block.type == "tool_use":
        tool_name = block.name
        tool_input = block.input
        result = get_weather(tool_input["city"])

```

`tools=tools` — this is the only new argument compared to Day 3: it tells the API “these are your available tools.”

`response.content` is a **list of blocks** — this time, instead of just text, one block might be `type == "tool_use"`, meaning the model decided it needs to call a function instead of answering directly.

`block.name` — which tool the model wants (`"get_weather"`).

`block.input` — the arguments the model filled in, as a dictionary, e.g. `{"city": "Jeddah"}`.

`get_weather(tool_input["city"])` — **your code**, not the model, actually runs the function and gets a real result.

```

python
messages.append({"role": "assistant", "content": response.content})
messages.append({
    "role": "user",
    "content": [{
        "type": "tool_result",
        "tool_use_id": block.id,
        "content": result
    }]
})

final_response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=500,
    tools=tools,
    messages=messages
)
print(final_response.content[0].text)

```

We append the model's tool request to `messages` first, so the conversation history stays consistent.

Then we append a `"tool_result"` message — this is how you hand the *result* of running the function back to the model. `tool_use_id` links this result to the specific request the model made (important if it asked for multiple tools at once).

We call the API **again** with the updated `messages` — this second call is where the model actually writes a natural-language answer using the real weather data you gave it.

This exact loop — *model requests tool* → *your code runs it* → *result goes back* → *model writes final answer* — is the pattern **every single one of your 10 BioInsight agents will follow**, just with different tools (parsing a FASTA file, computing statistics, searching literature, etc.) instead of `get_weather`.

Exercise

Write a second tool, `count_bases(sequence)`, that takes a DNA string like `"ATCGGCTA"` and returns a dictionary counting how many A's, T's, C's, and G's it contains. Wire it into the same loop above and ask "How many of each base are in ATCGGCTA?"

Day 5 — Section 1, Agent 1: The File Detective

Concept: agents that inspect real files

This is your first BioInsight agent. Its only job: look at an uploaded file and decide what kind of biological data it is (a sequence file, a tabular data file, or something else) so the right downstream agents can process it.

New library: reading file extensions and content

```
python
import os

def detect_file_type(filepath):
    extension = os.path.splitext(filepath)[1].lower()

    if extension in [".fasta", ".fa"]:
        return "sequence"
    elif extension in [".csv", ".tsv", ".xlsx"]:
        return "tabular"
    else:
        return "unknown"
```

`os.path.splitext(filepath)` — splits a filename into `(name, extension)`. E.g. `"data.csv"` becomes `("data", ".csv")`.

`[1]` — grabs just the extension part (index 1 of that tuple).

`.lower()` — converts to lowercase so `.CSV` and `.csv` are treated the same.

The `if/elif/else` chain routes the decision — this is deliberately simple logic (no AI needed) *before* we even involve the model, because you don't need an LLM to check a file extension. **Agent wisdom:** not every step needs to call the AI model — only use the model where judgment is genuinely needed.

Wrapping it as an agent tool

Now expose `detect_file_type` as a tool, exactly like `get_weather` on Day 4, so the model can call it as part of a conversation like "What kind of file is this: `sample.fasta`?" Reuse the same tool-loop pattern from Day 4 — only the function and its `tools` description change.

Exercise

Extend `detect_file_type` to also peek at the file's first line (using `open(filepath).readline()`) to double-check: FASTA files always start with `>`. Add this as a second validation layer before returning `"sequence"`.

Day 6 — Section 1, Agent 2: The Data Cleaner

Concept: why cleaning matters before analysis

Real data is messy — missing values, wrong types, duplicate rows. If you analyze messy data, you get misleading results. This agent's job: catch problems *before* they reach the analysis agents.

New library: pandas

```
powershell
pip install pandas
```

Pandas is the standard Python library for working with tabular data (rows and columns) — think of it as Excel, but in code.

```
python
```

```
import pandas as pd
```

```
def check_data_quality(filepath):
    df = pd.read_csv(filepath)

    missing_counts = df.isnull().sum()
    duplicate_rows = df.duplicated().sum()

    report = {
        "total_rows": len(df),
        "missing_values": missing_counts.to_dict(),
        "duplicate_rows": int(duplicate_rows)
    }

    return report
```

`import pandas as pd` — the `as pd` gives it a short alias so you type `pd.` instead of `pandas.` every time — this is a near-universal convention.

`pd.read_csv(filepath)` — reads a CSV file into a **DataFrame** (`df`), pandas' table object — rows and named columns, like a spreadsheet you can manipulate in code.

`df.isnull()` — returns a same-shaped table of `True/False`, marking every cell that's empty/missing.

`.sum()` — adds up the `True` values *per column* (True counts as 1), giving you a count of missing values in each column.

`df.duplicated()` — returns `True` for any row that's an exact repeat of an earlier row.

`.sum()` again — counts how many duplicate rows exist total.

We build a plain Python dictionary (`report`) summarizing the findings — this is the format we'll hand to the model so it can explain the results in plain language.

Exercise

Add a check for outliers in a numeric column: use `df["column_name"].mean()` and `df["column_name"].std()` to flag any value more than 3 standard deviations from the mean. This is the first real statistical concept you'll reuse heavily in Section 3.

Day 7 — Section 2: Sequence Analysis Agents

Concept: working with DNA/protein sequences

```
powershell
pip install biopython
```

Biopython is the standard library for biological sequence data — it knows how to read FASTA files, compute sequence statistics, and more, so you don't have to write parsers by hand.

Agent 3 — Sequence Reader

```
python
from Bio import SeqIO

def read_sequences(filepath):
    sequences = []
    for record in SeqIO.parse(filepath, "fasta"):
        sequences.append({
            "id": record.id,
            "length": len(record.seq),
            "gc_content": round(
                (record.seq.count("G") + record.seq.count("C")) / len(record.seq) * 100, 2
            )
        })
    return sequences
```

`from Bio import SeqIO` — imports Biopython's sequence-reading module.

`SeqIO.parse(filepath, "fasta")` — opens the file and reads it record-by-record, understanding FASTA's format (`>` header lines followed by sequence data) automatically.

`record.id` — the sequence's name/identifier from the `>` header line.

`record.seq` — the actual sequence data (like `"ATCGGCTA..."`), as a special Biopython object that supports operations like `.count()`.

GC content is a genuinely meaningful biology metric — the percentage of the sequence that's G or C bases, which relates to the sequence's stability. This is a real, useful calculation, not just a demo.

Agent 4 — Pattern Finder

```
python
def find_motif(sequence, motif):
    positions = []
    start = 0
    while True:
        index = sequence.find(motif, start)
        if index == -1:
            break
        positions.append(index)
        start = index + 1
    return positions
```

`sequence.find(motif, start)` — searches for `motif` (a short pattern, e.g. `"ATG"`) starting from position `start`. Returns `-1` if not found, otherwise the position where it starts.

The `while True` loop keeps searching *past* each found match (`start = index + 1`) so it finds every occurrence, not just the first.

This is a simplified version of what real bioinformatics motif-finding tools do — good enough to genuinely learn the concept.

Exercise

Run `read_sequences` on a small FASTA file (you can create one by hand — search "sample fasta file" for a tiny example, 2-3 sequences). Then use `find_motif` to search for the start codon `"ATG"` in one of the sequences and print all positions found.

Day 8 — Section 3: Statistical & Omics Analysis Agents

Agent 5 — Stats Analyst

```
python
def analyze_statistics(filepath, column):
    df = pd.read_csv(filepath)
    stats = {
        "mean": df[column].mean(),
        "median": df[column].median(),
        "std_dev": df[column].std(),
        "min": df[column].min(),
        "max": df[column].max()
    }
    return stats
```

`df[column]` — selects one column from the table by name, returning a pandas **Series** (a single column of data).

`.mean()`, `.median()`, `.std()`, `.min()`, `.max()` — built-in pandas methods computing standard descriptive statistics over that column in one line each, instead of writing loops by hand.

Why this matters for biological data: e.g. in gene expression data, comparing the mean expression level of a gene across samples tells you whether it's typically high or low, and the standard deviation tells you how much it varies.

Agent 6 — Anomaly Spotter

```
python
```

```
def find_outliers(filepath, column):
    df = pd.read_csv(filepath)
    mean = df[column].mean()
    std = df[column].std()

    outliers = df[(df[column] - mean).abs() > 3 * std]
    return outliers[[column]].to_dict(orient="records")
```

`(df[column] - mean).abs()` — subtracts the mean from every value, then takes the absolute value — this gives "how far is each value from average," ignoring direction.

`> 3 * std` — flags any value more than 3 standard deviations away, a common statistical threshold for "this is unusual."

`df[...]` with a condition inside — this is pandas' filtering syntax: it returns only the rows where the condition is `True`.

`.to_dict(orient="records")` — converts the filtered table into a list of dictionaries (one per row) — an easy format to hand to the AI model or display in the dashboard later.

Exercise

Create a small CSV by hand (5-6 rows, one numeric column like `expression_level`) with one obviously extreme value. Run both functions on it and confirm `find_outliers` correctly flags just that one row.

Day 9 — Section 4: Research Assistant Agents

Agent 7 — Literature Scout

This agent's "tool" isn't a local Python function — it's a **web search**. You'll wire this the same way as Day 4's `get_weather`, except the tool function actually calls out to a real search API instead of returning a fixed string. (When we wire this into the real platform on Day 12, this reuses tools available directly in your agent code — no separate library needed if you're calling it through your own backend, since your backend can make search requests itself.)

Agent 8 — Summarizer

```
python
def summarize_findings(raw_text):
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=300,
        system="Summarize biological research findings in plain, non-technical language, in 3 sentences maximum.",
        messages=[{"role": "user", "content": raw_text}]
    )
    return response.content[0].text
```

Notice this agent doesn't need a custom tool at all — it's a **pure LLM call** with a very specific `system` prompt. Not every agent needs tool use; some agents just need the *right instructions* to transform text usefully.

Keeping `max_tokens=300` forces a concise answer — a good habit for summarizer agents, so downstream agents (or the user) aren't flooded with text.

Exercise

Take any 2-3 paragraph piece of text about a biology topic (copy your own summary of a topic you know, in your own words) and run it through `summarize_findings`. Compare the length and tone of the output to the input.

Day 10 — Section 5: Reporting Agents

Agent 9 — Chart Maker

```
powershell
pip install matplotlib
```

```
python
import matplotlib.pyplot as plt

def make_bar_chart(data_dict, title, output_path):
    plt.figure()
    plt.bar(data_dict.keys(), data_dict.values())
    plt.title(title)
    plt.savefig(output_path)
    plt.close()
```

`import matplotlib.pyplot as plt` — `pyplot` is matplotlib's charting module; `plt` is the standard alias.

`plt.figure()` — starts a new, blank chart canvas.

`plt.bar(keys, values)` — draws a bar chart: dictionary keys become the x-axis labels, values become bar heights.

`plt.savefig(output_path)` — saves the chart as an image file (e.g. `"chart.png"`) instead of just displaying it — important because your dashboard will need image *files* to show.

`plt.close()` — frees up memory by closing the figure. Easy to forget, but matters if you're generating many charts in a loop (as your platform will).

Agent 10 — Report Writer

```
python
def write_report(all_results):
    summary_text = json.dumps(all_results, indent=2)
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=800,
        system="You are a report-writing assistant. Combine these biological analysis results into one clear, well-organized report with headers.",
        messages=[{"role": "user", "content": summary_text}]
    )
    return response.content[0].text
```

`json.dumps(all_results, indent=2)` — converts a (possibly nested) Python dictionary of *all* your agents' results into a readable JSON string.

`indent=2` just makes it human-readable with line breaks, purely for clarity.

This agent is your platform's "finale" — it's the one that takes everything the other 9 agents produced and writes the final human-readable output.

Exercise

Manually create a small dictionary combining fake results from 2-3 "agents" (e.g. `{"sequence_stats": {...}, "outliers": [...]}`) and run it through `write_report`. Read the output — does it organize the information sensibly?

Day 11 — How Agents Talk to Each Other

Concept: orchestration

So far each agent has run alone. Today you learn the **orchestrator pattern**: one piece of code (not an AI model, just plain Python) that decides which agent runs when, and passes each agent's output into the next agent's input.

```
python
```

```
def run_pipeline(filepath):
    file_type = detect_file_type(filepath)    # Agent 1

    if file_type == "sequence":
        cleaned = filepath                    # sequences don't need pandas cleaning
        sequences = read_sequences(cleaned)    # Agent 3
        motifs = find_motif(str(sequences[0]), "ATG") # Agent 4
        section_results = {"sequences": sequences, "motifs": motifs}

    elif file_type == "tabular":
        quality_report = check_data_quality(filepath) # Agent 2
        stats = analyze_statistics(filepath, "value") # Agent 5
        outliers = find_outliers(filepath, "value") # Agent 6
        section_results = {
            "quality": quality_report,
            "stats": stats,
            "outliers": outliers
        }

    report = write_report(section_results)    # Agent 10
    return report
```

This function is the **spine of your whole platform** — notice it contains *no AI calls itself* except the final report step. It's just Python logic deciding "if it's a sequence file, run these agents; if tabular, run those agents."

Each variable (`sequences`, `stats`, `outliers`..) is one agent's output becoming the *input* to a later step — this is literally what "agents working together" means in practice: structured data passed hand-to-hand, not agents mysteriously "talking" in English to each other.

Exercise

Trace through the function by hand with a sequence file: write down, on paper or in a comment, exactly which agents run and what each one receives as input and produces as output. This mental model is essential before you wire it to a UI.

Days 12–13 — Full Backend + Dashboard

Backend: exposing the pipeline over HTTP

```
powershell
pip install fastapi uvicorn python-multipart
```

```
python
from fastapi import FastAPI, UploadFile
import shutil

app = FastAPI()

@app.post("/analyze")
async def analyze(file: UploadFile):
    filepath = f"uploads/{file.filename}"
    with open(filepath, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    report = run_pipeline(filepath)
    return {"report": report}
```

`UploadFile` — a FastAPI type that handles file uploads from a web request properly (streaming, not loading the whole file into memory at once).

`shutil.copyfileobj(file.file, buffer)` — copies the uploaded file's contents into a real file on your disk, so the rest of your pipeline (which expects a filepath) can read it normally.

`run_pipeline(filepath)` — this is Day 11's orchestrator function, now triggered by a real HTTP upload instead of you calling it manually in a script.

Run it:



```
powershell
uvicorn main:app --reload
```

Dashboard: Streamlit file upload UI



```
python
import streamlit as st
import requests

st.title("BioInsight")

uploaded_file = st.file_uploader("Upload a biological data file")

if uploaded_file is not None:
    files = {"file": (uploaded_file.name, uploaded_file.getvalue())}
    response = requests.post("http://127.0.0.1:8000/analyze", files=files)
    st.write(response.json()["report"])
```

`st.file_uploader(...)` — renders a real file-upload widget in the browser.

`files = {"file": (...)}` — this is the specific format the `requests` library expects for sending file uploads over HTTP — the key `"file"` must match the parameter name (`file: UploadFile`) in your FastAPI endpoint.

`requests.post(url, files=files)` — sends the uploaded file to your backend and waits for the JSON response.

`st.write(...)` — displays the final report text in the dashboard.

Exercise

Run both the backend (`uvicorn`) and dashboard (`streamlit run`) at the same time, in two terminals. Upload a small CSV or FASTA file through the dashboard and confirm you get a written report back.

Day 14 — Integration, Testing, and What's Next

Checklist for a working end-to-end platform

- ☐ Upload a tabular CSV → confirm quality check, stats, and outliers all show up in the report
- ☐ Upload a FASTA file → confirm sequence stats and motif search show up in the report
- ☐ Try an unsupported file type → confirm it fails gracefully instead of crashing
- ☐ Push the whole project to GitHub (reuse your Day 7 git knowledge from the original plan)

What real platforms add next (not required now, just context)

- Persistent storage** — a real database instead of re-analyzing from scratch each time
- Async/parallel agents** — running independent agents (like Sequence Reader and Stats Analyst) at the same time instead of one after another
- Agent frameworks** — LangGraph or CrewAI replace your hand-written orchestrator with more powerful routing, retries, and multi-step planning
- The actual literature search** — wiring Agent 7 to a real search API (like PubMed's E-utilities) instead of a placeholder

By the end of Day 14, you won't just have copied code — you'll understand *why* each of the 10 agents exists, what each library call actually does, and how to extend any section with new agents on your own.